Member 3 — Communication and Backend

Member placeholder: [MEMBER 3 NAME]

Proposed viva responsibility: USB serial acquisition, [JSON] packet parsing, strict schema/status/range validation, duplicate and stale handling, SSE/HTTP APIs, storage handoff and local error behavior.

Authorship disclaimer: this document maps the repository's backend capabilities to a proposed HOD responsibility. It does not prove that [MEMBER 3 NAME] authored telemetry_bridge.py. The team must confirm individual history; use **"the team implemented"** for source-backed shared work.

A. Responsibility scope

A.1 One-sentence scope

For this discussion, [MEMBER 3 NAME] is responsible for explaining how the local Python bridge turns a raw USB serial line into a validated, fresh, fail-closed telemetry view — and how it serves that view over SSE/HTTP without inventing or replaying data.

A.2 Included in this responsibility

- telemetry_bridge.py process startup and local bind.
- Serial port discovery and the serial reader thread.
- [JSON] prefix handling, newline framing and buffer safety.
- DuplicateSuppressor (identical serial lines) versus duplicate JSON keys.
- parse_packet() and _object_without_duplicates().
- _validate_packet() and its per-domain validators:
  - _validate_measurement();
  - _validate_temperature();
  - _validate_fever();
  - _validate_imu().

- Sensor presence/active/active_max consistency.
- Actual TMP117 versus clinical wrist validation.
- Core null/fever not_available enforcement.
- Freshness (STALE_SECONDS = 5), snapshot(), set_disconnected() and non-replay behavior.
- SSE /api/stream, queue behavior and slow-client policy.
- HTTP handlers: /api/telemetry, /api/history, /api/baseline, /api/events, static files.
- POST baseline origin/size/device checks.
- Storage/baseline handoff and nonfatal storage-error reporting.
- Local-only security posture and its limitations.

A.3 Outside primary scope — but required knowledge

- Firmware producer/status semantics: Member 1.
- TMP117 protocol/diagnostic and physical verification: Member 2.
- SQLite schema/history queries and dashboard DOM: Member 4.
- Temperature safety policy, baseline statistics and alert rules: Member 5.

Member 3 must know the exact temperature semantics because the bridge is where many contradictions are rejected.

A.4 Key files

File	Role
telemetry_bridge.py	746-line local bridge: serial, validation, SSE/HTTP, storage handoff
telemetry_store.py	Storage target; repeats wrist/raw checks in measurements()
temperature_safety.py	Shared policy constants, including the exact fever message
personal_baseline.py	Baseline comparisons/events called after acceptance

File	Role
tests/test_telemetry.py	Parser/schema/status/temperature/fever/stale tests
tests/fixtures.py	Synthetic packets; not clinical data
README.md	Telemetry contract, local API, security and verification status

B. Achievements attributable to this area

Achievement	What the repository contains	Evidence
Serial-to-SSE pipeline	Reader thread, newline framing, strict parse, live snapshot, SSE	[SOURCE] serial_reader_thread(), snapshot(), /api/stream
Duplicate protection	Identical serial line suppression in 1.5 s plus duplicate JSON-key rejection	[SOURCE] DuplicateSuppressor, _object_without_duplicates(); [TEST]
Strict schema 2 validation	Device/schema/sensor/status/range/timestamp checks	[SOURCE] _validate_packet(); [TEST]
Temperature defense in depth	Raw conversion, stuck-pattern, range, actual/wrist matching, unverified null rule	[SOURCE] _validate_temperature(); [TEST]
Fever/core enforcement	Core null; fever not_available, exact message, no rule	[SOURCE] _validate_fever(); [TEST]
Freshness/no replay	5 s stale timeout; snapshot() returns null telemetry; new browser not shown old packet	[SOURCE] snapshot(); [TEST]
Local APIs	Telemetry, SSE, history, baseline, events, static assets	[SOURCE] TelemetryRequestHandler
Storage handoff	Valid packet accepted, recorded and compared; storage failure visible but nonfatal	[SOURCE] _accept_packet()
Local-only security baseline	Bind 127.0.0.1; same-origin POST check; bounded	[SOURCE] main(), do_POST()

Achievement	What the repository contains	Evidence
	request size	
Reproducible host evidence	62 Python and 10 Node tests passed, including backend cases	[TEST] fresh 2026-10-29

B.1 What this area does not establish

- That the serial device exists, is the correct one, or is electrically reliable.
- That firmware/sensor readings are accurate or clinically valid.
- That the bridge is secure enough for an untrusted network or real patient data.
- That SSE/HTTP behavior has been load-tested with many clients.
- That baseline/statistics are clinically validated.
- That `pyserial` is installed on every deployment machine.

C. Detailed code walkthrough

C.1 Process startup — `main()`

`main()` (near the end of `telemetry_bridge.py`):

1. Parses optional HTTP port (default `8080`), `--serial-port`, and `--database`.
2. Creates `TelemetryStore(args.database)`; default `data/telemetry.sqlite3`.
3. Creates `PersonalBaseline(telemetry_store)`.
4. Builds a `partial(TelemetryRequestHandler, directory=<repo root>)`.
5. Starts `ThreadingHTTPServer(('127.0.0.1', args.port), handler)`.
6. Starts `serial_reader_thread` as a daemon thread.
7. Prints the dashboard URL and serves forever until KeyboardInterrupt.

The bind address is explicitly `127.0.0.1`, not `0.0.0.0`. This is a local prototype boundary, not an authentication mechanism.

C.2 Serial discovery and reader

find_serial_port():

ports = [p.device for p in list_ports.comports() if p.vid is not None]

return ports[0] if len(ports) == 1 else None

- Only USB-style ports with a VID are considered.
- Exactly one must be present; zero or multiple results return None.
- An explicit --serial-port avoids guessing.
- Only one program may own the COM port; a running pio device monitor conflicts.

serial_reader_thread(port=None) loop:

1. Uses the explicit port or re-discovers one.
2. Sets hardware_status['port'].
3. If no unique port, sets a disconnected error and retries after 2 s.
4. Opens serial.Serial(port=None, baudrate=115200, timeout=0.5), dtr=False, rts=False; sets exclusive=True on POSIX.
5. Publishes "Serial port open; waiting for ESP32 telemetry."
6. Accumulates bytes in a buffer (capped at 65,536 bytes).
7. Splits on newline, decodes UTF-8 with replacement, strips.
8. Applies duplicate_suppressor.duplicate(text); skips and increments duplicates_skipped.
9. Calls parse_packet(text); increments invalid_packets when a line resembling an object is rejected.
10. Calls _accept_packet(packet) on success.
11. If connected but no fresh packet for 5 s, calls set_disconnected(...).
12. On serial exception, closes, prints, waits 2 s and retries.

The buffer prevents a partial line from being parsed; the 65,536-byte cap prevents unbounded memory growth from a corrupt stream.

C.3 Duplicate handling — two different problems

Identical serial lines: DuplicateSuppressor(window=1.5) suppresses an identical text only if it arrives within 1.5 s of the last occurrence. A repeat after the window is accepted. This handles a serial driver/bridge repeating the same packet, not two genuinely identical frames far apart.

Duplicate JSON keys: _object_without_duplicates(pairs) raises on a repeated key, causing parse_packet() to return None. This prevents {"heart_rate": 72, "heart_rate": 180} from silently keeping the last value.

These are not interchangeable. A test can exercise the 1.5 s line gate independently of duplicate keys.

C.4 Parsing — parse_packet()

```
def parse_packet(line):
    text = line.removeprefix('[JSON]').strip()
    if not text.startswith('{'):
        return None
    try:
        packet = json.loads(
            text, object_pairs_hook=_object_without_duplicates)
    except (ValueError, TypeError):
        return None
    return packet if _validate_packet(packet) else None
```

Precise behavior:

- The firmware emits a [JSON] prefix; the parser removes it if present.
- Because it also accepts a bare object beginning {, the prefix is a producer convention rather than a parser-enforced requirement.
- JSON arrays, boot text, truncated lines and empty objects fail.
- Duplicate keys fail during decoding.
- The parsed packet must then pass _validate_packet().

- _json_safe() recursively rejects NaN/Infinity and non-string dictionary keys before acceptance.

C.5 Packet validation — _validate_packet()

The validator is intentionally strict and returns a boolean; any failure rejects the entire packet rather than partially updating state.

It checks:

1. Packet is a dictionary and JSON-safe.
2. device_id is a non-empty string at most 128 characters.
3. schema_version == 2.
4. uptime_s is finite and non-negative.
5. vitals, sensors, alerts are dictionaries.
6. alerts.sos and alerts.fall are booleans.
7. sensors.max, sensors.mpu, sensors.tmp117 are booleans.
8. If present, oled, sht40, mlx are booleans.
9. sht40 or mlx true is rejected.
10. active_max == 3.
11. active equals the count of true installed sensors.
12. finger_on is boolean.
13. HR, SpO2, motion pass _validate_measurement().
14. Temperature passes _validate_temperature().
15. Core value is null; core status is not_available.
16. Legacy temperature aliases are null/absent.
17. Fever passes _validate_fever().
18. IMU nested data passes _validate_imu().
19. Generic numeric fields are finite and within ranges.

20. Timestamps for valid optical/motion values are non-negative integers.

21. String fields have the right type; `steps` is an integer; `urgent` is boolean.

22. `board.flash_bytes`/`psram_bytes` are non-negative integers if present.

The bridge does not require every optional field the firmware emits, but it rejects any present field that contradicts the contract. The core status and other safety fields are treated as non-negotiable.

C.6 Measurement validation

`_validate_measurement(vitals, value_key, status_key, low, high, sensor, sensors, optical=False)` enforces:

- status is a recognized string in `SENSOR_STATUSES`;
- a non-null value is a finite number in range;
- `valid` requires the sensor present and a value;
- optical `valid` also requires `finger_on is True`;
- a non-valid status requires `value is None`.

The per-domain calls use:

Metric	Sensor	Bridge range
Heart rate	max	20..250 bpm
SpO2	max	0.1..100 %
Motion	mpu	0..20 g

These differ slightly from firmware acceptance windows (for example, firmware accepts HR 30..240), so both layers use conservative engineering bounds rather than one being a copy of the other.

C.7 Temperature validation — `_validate_temperature()`

This is the most important backend function for the project's clinical honesty.

Actual TMP117:

- status must be an actual-sensor status (`SENSOR_STATUSES` minus `unverified`);

- if valid: sensors.tmp117 true, finite value, raw code 0..0xFFFF, not 0x0000/0xFFFF, non-negative timestamp, value in -55..150, signed conversion within 0.02 °C;
- if non-valid: value/raw/timestamp must all be null; if the sensor is absent, status must be not_available.

Clinical wrist:

- wrist_contact_status must be verified or unverified;
- wrist status must be a recognized status;
- if unverified: wrist status must be unverified, with null value/raw/timestamp;
- if verified and status valid: TMP117 must be valid, raw must match actual raw, timestamp must match actual timestamp, value must match actual value within 0.02 °C, raw must convert within 0.02 °C, and wrist must be -10..60 °C;
- non-valid verified wrist data must be null.

Current firmware outcome: actual TMP117 can be valid; wrist_contact_status is unverified; wrist value/raw/timestamp are null. The bridge accepts this exact pattern and rejects a wrist value without verified contact.

The bridge does **not** compute core temperature and does **not** apply a fixed offset.

C.8 Fever validation — _validate_fever()

If vitals.fever_screening is absent, validation tolerates it. If present, it must:

- be a dictionary;
- have status == "not_available";
- have screening_supported absent/false (never true);
- have the exact message **Core temperature unavailable — confirmation required**;
- have rule is None;
- if an input object exists:
  - measurement wrist_skin_temperature (or absent);
  - unit C (or absent);

- ○ quality equal to the wrist status;
- ○ measured boolean;
- ○ if measured, verified contact + valid wrist + matching value/timestamp/raw and non-stuck raw;
- ○ if unmeasured, no value/timestamp/raw and wrist status not valid.

This prevents a producer from smuggling a fever rule or a "measured" wrist input into an otherwise null clinical packet.

C.9 Acceptance and storage handoff — _accept_packet()

On a valid packet:

1. Adds packet['_received_at'] = time.time().
2. Under state_lock, sets latest_telemetry, updates last_packet_time (monotonic), increments packets_received, sets connected=True, clears the error, records device_id.
3. Calls TelemetryStore.record(packet) if storage exists.
4. Calls PersonalBaseline.compare(packet) and records new SOS/fall events.
5. On sqlite3.Error, sets hardware_status['storage_error'] but keeps the live packet.
6. Calls broadcast().

Consequences:

- ● A storage failure does not fabricate a reading; the dashboard can still show live data but must display the storage error.
- ● Events are only inserted when the alert transitions from inactive to active (last_alerts).
- ● latest_comparisons may be {} if no baseline exists or values are unavailable.
- ● Invalid packets never reach this function.

C.10 Live snapshot and stale policy

snapshot():

- ● copies hardware_status;

- considers data fresh only if last_packet_time exists and now - last_packet_time < 5.0;
- if not fresh and status says connected, flips connected=False with a timeout error;
- returns telemetry only if fresh and connected;
- returns comparisons only if fresh and connected.

set_disconnected() clears latest_telemetry and last_packet_time, sets connected false, and broadcasts. A new browser therefore cannot be shown a stale last packet. The backend test test_stale_packet_is_not_replayed_to_new_browser covers this.

C.11 SSE — /api/stream

- The handler responds with text/event-stream, no-cache.
- Each client gets queue.Queue(maxsize=8).
- The initial payload is snapshot().
- It writes data: <json>₩n₩n, flushes, then waits up to 1 s for the next payload; on timeout it sends a fresh snapshot.
- broadcast() uses put_nowait; if a client queue is full, it drops the oldest item and inserts the newest. A slow browser cannot block serial acquisition.
- On disconnect, the client is removed in finally.

This is a live-view convenience, not a durable delivery guarantee. History is stored separately.

C.12 HTTP APIs

Route	Behavior
GET /api/telemetry	snapshot()-style response with hardware status and latest telemetry
GET /api/stream	SSE live stream
GET /api/history	Requires store; parses seconds as integer; returns TelemetryStore.history(); 503/400 on failures
GET /api/baseline	Requires baseline; optional device_id; returns summary

Route	Behavior
GET /api/events	Requires baseline; optional device_id; returns up to 30 events
POST /api/baseline	Start/end collection with strict JSON/size/device checks
Static routes	/, /index.html, /dashboard.html, /wiring-guide.html, assets/fonts

send_json() uses json.dumps(..., allow_nan=False) and Cache-Control: no-store. Unknown paths return 404.

C.13 POST baseline protection

do_POST() requires exactly /api/baseline and an initialized baseline service. It:

- rejects a cross-origin Origin whose netloc differs from Host with 403;
- requires Content-Length in 1..4096;
- parses a JSON object;
- requires device_id equal to the currently connected device;
- for start: requires the device connected and a string person; calls PersonalBaseline.start();
- for end: calls .end();
- unknown action or bad body → 400.

This is localhost protection against accidental cross-origin dashboard requests, not a complete authentication/authorization system.

D. Technical decisions and trade-offs

Decision	Reason	Trade-off
Reject the whole packet on any contradiction	A partially accepted patient packet is worse than no update	A minor producer bug hides all fields until fixed; diagnostics must be clear
Strict schema/status/range validation	Prevents false values from reaching UI/store/baseline	More maintenance when the schema evolves
Duplicate JSON-key rejection	Prevents parser last-key-wins ambiguity/security issues	Some JSON producers with duplicate keys are rejected

Decision	Reason	Trade-off
Identical-line suppression in 1.5 s	Handles serial repeats without dropping deliberate later frames	A genuinely repeated packet within 1.5 s is skipped
Five-second freshness	Bounds stale display; faster than baseline/history cadence	Short Wi-Fi/USB hiccups can show offline
Local bind only	Avoids exposing an unauthenticated prototype to a network	Not usable for genuine remote monitoring without added security
One unique USB port	Avoids guessing among devices	Zero/multiple USB serial devices require explicit selection
SSE queue of 8 with drop-oldest	Slow browsers cannot block acquisition	Slow clients miss intermediate frames; only latest state matters
Storage failure nonfatal	Live view remains useful if disk is full/locked	User must notice `storage_error`; data may be missing
No fake/demo data	Protects evidence integrity	Empty dashboard when hardware is absent
Separate bridge validation from storage/dashboard checks	Defense in depth	Three implementations must stay consistent
No authentication/TLS	Appropriate only for localhost demo	Not safe for real patient data or open network

D.1 Why the bridge must not “fix” packets

It would be easy to coerce `"72"` to 72, turn `NaN` into `null` or copy actual TMP117 into wrist. Each would make the demo look better and the evidence worse. The bridge rejects or masks instead:

- no numeric coercion of strings/booleans;
- no NaN/Infinity acceptance;
- no actual-to-wrist promotion;
- no core/fever invention;

- no stale replay;
- no optional sensor invention.

E. Cross-module contributions and interfaces

E.1 Firmware → bridge

The bridge expects the firmware's schema-2 structure and uses the status/raw/timestamp evidence to validate:

- installed-sensor booleans and active count;
- optical values and finger contact;
- motion/IMU data;
- actual TMP117 raw/value/timestamp;
- unverified or verified wrist contact;
- core null and fever policy;
- alerts and summary.

If the firmware changes a field/status, the bridge schema and tests must change deliberately.

E.2 Bridge → storage

`_accept_packet()` calls `TelemetryStore.record()` only after validation. The store repeats wrist/raw checks in `measurements()`, including verified contact, matching actual evidence, range and raw conversion. This redundancy is intentional because storage is a separate trust boundary.

E.3 Bridge → dashboard

- `/api/stream` payload contains `status`, `telemetry`, `comparisons`.
- `/api/history` returns bucketed points and `bucket_seconds`.
- `/api/baseline`/`/api/events` power the baseline page.
- Dashboard code consumes `_received_at` for duplicate live updates, and displays storage/connection errors.
- The dashboard independently rechecks metric status and temperature evidence.

- The exact labels are **Wrist Skin Temperature (°C)** for the clinical card and **TMP117 Sensor Temperature (°C)** for the actual sensor. Core is null/not_available; fever is not_available with the exact message **Core temperature unavailable — confirmation required**.

E.4 Bridge → safety/baseline

- Temperature safety is applied both in firmware/validator and in temperature_safety.py when the terminal engine summarizes a packet.
- Baseline comparisons are computed after acceptance, but use measurements(); therefore unverified wrist values cannot feed comparisons.
- Event insertion is tied to alert transitions, not repeated every packet.

E.5 Integration contribution to state clearly

"My backend responsibility was to make the serial path strict and observable: validate the whole contract, refuse contradictions, expire stale data, and serve only fresh accepted packets. The bridge does not create clinical values; it rejects or exposes their absence."

Do not claim authorship of the entire backend unless confirmed.

F. Evidence classification for this area

Claim	Classification	Verification
Serial reader parses [JSON]/newline packets and reconnects	[SOURCE]	serial_reader_thread()
Duplicate lines and duplicate keys are handled separately	[SOURCE] + [TEST]	DuplicateSuppressor, _object_without_duplicates; test_telemetry.py
Schema/status/range validation is strict	[SOURCE] + [TEST]	_validate_packet() and tests
Actual/wrist/core/fever rules enforced	[SOURCE] + [TEST]	_validate_temperature(), _validate_fever()
Stale packet is not replayed	[SOURCE] + [TEST]	snapshot(), test_stale_packet_is_not_replayed_to_new_browser
SSE/API routes exist	[SOURCE]	TelemetryRequestHandler

Claim	Classification	Verification
Local-only bind	[SOURCE]	ThreadingHTTPServer(('127.0.0.1', ...))
Storage failure is visible but nonfatal	[SOURCE]	_accept_packet() sqlite3.Error branch
62 Python tests pass	[TEST] fresh 2026-10-29	python -m pytest -q
Real serial/USB operation	[UNVERIFIED HW]	No device/capture in this pass
Multi-client/load/security testing	**Not performed**	No load/security test evidence

F.1 What the tests cover and what they do not

The synthetic fixtures cover protocol shape, malformed types, wrong schema/active counts, contradictions, stuck raw, raw mismatch, missing raw/timestamp, offline sensor, wrist/contact rules, fever text, duplicates and stale replay. They do not prove:

- the physical serial link or driver behavior;
- throughput/latency under load;
- browser SSE reconnection across network failures;
- database corruption/disk-full recovery beyond the surfaced storage_error;
- authentication, TLS or multi-user safety;
- clinical interpretation.

G. Debugging: backend scenarios

G.1 “No serial port found”

- find_serial_port() returns only a single USB VID port; zero/multiple → None.
- Check pio device list.
- Close pio device monitor; only one process can own the port.
- Pass --serial-port COMx deliberately; do not copy another machine’s port.
- If pyserial is missing, the bridge reports that and retries.

G.2 “Packets arrive but the dashboard says offline”

- Inspect `/api/telemetry`: hardware status includes `connected`, `error`, `packets_received`, `invalid_packets`, `duplicates_skipped`.
- If `invalid_packets` is increasing, compare the serial line with the schema; do not disable validation.
- If `packets_received` is not increasing, check `[JSON]` framing/newline and serial ownership.
- After 5 s without a fresh packet, `snapshot()` intentionally reports offline and null telemetry.

G.3 “A packet with valid data is rejected”

Likely contradictions:

- `active`/`active_max` wrong;
- sensor false but status `valid`;
- status non-valid but value non-null;
- raw `0x0000`/`0xFFFF`;
- raw/value conversion difference > `0.02`;
- wrist value without verified contact or mismatched actual evidence;
- non-null core;
- fever rule/status/message mismatch;
- duplicate JSON key;
- timestamp missing for a valid value;
- NaN/Infinity anywhere.

Use the test fixtures and validator functions to isolate the exact field. Fix the producer or explicitly version the schema; never weaken the safety checks silently.

G.4 “Duplicate lines/values”

- Repeated serial text within 1.5 s increments `duplicates_skipped`.
- Duplicate JSON keys cause parse failure (not duplicate suppression).

- Repeated values in separate packets are not necessarily duplicates; the bridge does not deduplicate by measurement value.

G.5 "Storage error"

_accept_packet() catches sqlite3.Error, stores hardware_status['storage_error'], but still updates the live view. Check disk path/permissions/free space; the dashboard should surface the error. Do not claim history is complete while the error persists.

G.6 "SSE client lags"

Each client has an 8-slot queue and drop-oldest behavior. A slow client loses intermediate frames; it should still receive periodic snapshots. This protects serial acquisition. For historical completeness, use /api/history, not SSE.

G.7 "POST baseline rejected"

- Origin/Host mismatch → 403.
- Content length not 1..4096 → 400.
- Body not an object/missing fields → 400.
- Device ID not the currently connected device → 400.
- Start while disconnected → 400.
- Wrong path → 404.

This is a local dashboard control, not a public API.

H. Testing and verification

H.1 Fresh checks (2026-10-29)

Check	Result
python -m pytest -q	62 passed
node --test tests/dashboard_core.test.js	10 passed
Ruff, Python compilation, JS syntax	passed
Main firmware build	SUCCESS
TMP117 diagnostic build	SUCCESS

H.2 Backend-relevant tests

tests/test_telemetry.py includes cases for:

- complete contract acceptance;
- non-telemetry/non-finite rejection;
- wrong schema/active count;
- contradictory status/value;
- core/fever rejection;
- stuck-bus raw marked valid;
- NaN in nested unknown fields;
- uninstalled optional sensors/fever rules;
- fever input contradicting wrist observation;
- raw mismatch;
- valid actual without raw/timestamp;
- offline sensor with valid status;
- actual status unverified rejection;
- actual outside clinical range but valid;
- unverified wrist with non-null data;
- verified wrist matching actual;
- duplicate temperature keys;
- non-null legacy alias;
- duplicate suppressor timing;
- stale packet not replayed;
- temperature semantics and exact fever message.

tests/test_history.py covers store-side invalid/missing/zero, unverified wrist rejection, valid verified storage, raw requirement, persistence/device isolation and gaps.

H.3 What to rerun after a backend change

1. `python -m pytest -q`.
2. `python -m ruff check ...`, Python compile, JS syntax checks.
3. Both PlatformIO builds if the schema/producer contract changes.
4. A synthetic fixture pass through `parse_packet()`.
5. If hardware is available: serial capture, stale/reconnect test, storage failure test, dashboard SSE test.

H.4 What tests do not prove

They are host-side synthetic tests. They do not prove real serial framing, USB driver stability, multi-client SSE performance, security, disk-full recovery or clinical validity.

I. HOD preparation

I.1 60–90 second explanation

"My allocated area is the communication and backend layer. The ESP32 sends one-line JSON at 115200 baud with a `[JSON]` prefix. The bridge reads complete serial lines, suppresses identical repeats within 1.5 seconds, rejects duplicate JSON keys, and then validates the whole schema-2 packet. It checks sensor presence, active count, status/value consistency, numeric ranges, timestamps, actual TMP117 raw conversion, the unverified or verified wrist rules, core null, and the exact fever policy. Only a valid packet updates live state and reaches SQLite and baseline comparison. The live view expires after five seconds, so a new browser never receives a stale packet. The bridge serves SSE at `/api/stream`, history, baseline, events and telemetry APIs, binding only to localhost. A storage failure is surfaced but does not fabricate telemetry. The 62 Python tests pass and the builds succeed, but the real serial link, load behavior, security and sensor accuracy have not been validated."

I.2 Technical questions and answers

1. **What is the bridge's main job?** Read the ESP32 serial stream, validate complete schema-2 packets, expose fresh telemetry over SSE/HTTP, hand valid packets to storage/baseline, and never fabricate or replay data.

2. **How are serial lines framed?** Bytes are buffered and split on newline. Partial lines stay in the buffer; the buffer is capped at 65,536 bytes. UTF-8 decode uses replacement, then the line is stripped and parsed.

3. **Does the parser require the [JSON] prefix?** The firmware emits it, and the parser removes it if present. A bare JSON object beginning { is also accepted, but it must pass full validation. Non-object/boot text is rejected.

4. **What is the difference between DuplicateSuppressor and duplicate-key rejection?** The suppressor drops identical serial text repeated within 1.5 s. Duplicate-key rejection catches ambiguous keys inside one JSON object via object_pairs_hook; it is a correctness/security rule, not a timing rule.

5. **Why reject the entire packet rather than just one bad field?** A partially trusted patient packet could mix valid and invalid evidence. Whole-packet rejection is fail-closed; the producer must fix the contract or a schema change must be deliberate.

6. **What does _validate_packet() check first?** It checks the object is JSON-safe, device ID, schema 2, uptime, the three required dictionaries, alerts booleans, installed sensor booleans, absent SHT40/MLX, active count/active_max, and finger state before metric validators.

7. **How is the TMP117 actual reading validated?** Status must be an actual-sensor status; valid requires sensor present, value in -55..150, raw 0..0xFFFF not 0/0xFFFF, timestamp, and signed conversion within 0.02 °C. Non-valid requires all evidence null.

8. **How is the clinical wrist value validated?** With unverified contact, status must be unverified and value/raw/timestamp null. With verified, a valid value must match the actual TMP117 value/raw/timestamp, convert within 0.02 °C, and be in -10..60 °C. Current firmware never sends verified.

9. **What are the exact core/fever rules?** Core value must be null and status not_available. Fever must be not_available, must not support screening, must have rule: null, and must use the exact message **Core temperature unavailable — confirmation required**.

10. **Why does the bridge expire telemetry after five seconds?** To prevent stale data from being shown as live. snapshot() returns null telemetry unless the last packet is younger than STALE_SECONDS = 5. This is separate from firmware sensor staleness thresholds.

11. **How does SSE handle slow browsers?** Each client has a queue of 8; broadcast() drops the oldest item when full, so a slow browser cannot block acquisition. It is latest-state delivery, not guaranteed delivery of every packet.

12. **What endpoints exist?** /api/telemetry, /api/stream, /api/history, /api/baseline (GET/POST), /api/events, plus static dashboard assets, on 127.0.0.1:8080 by default.

13. **What happens if SQLite fails?** _accept_packet() catches sqlite3.Error, sets hardware_status['storage_error'], and still broadcasts live telemetry. The dashboard is expected to show the storage error; history completeness is not claimed.

14. **How is the baseline POST protected?** Localhost bind, same-origin check (Origin/Host), request size 1–4096 bytes, JSON object, current connected device ID, known action. This is not full authentication.

I.3 Five deeper follow-up questions

1. **"Why not coerce strings to numbers to be tolerant?"** Coercion hides producer bugs and can create false measurements. The bridge rejects non-numeric types; the schema is a contract, not a suggestion.

2. **"What is the trust boundary between bridge and SQLite?"** They are separate layers: the bridge validates the wire packet, and TelemetryStore.measurements() independently rechecks status/raw/wrist evidence before storing. This defense in depth reduces the chance of a UI-only fix or a single coding error.

3. **"Could a valid packet still be clinically misleading?"** Yes. Schema validity is not clinical validity. Values may be inaccurate, the wrist value may be unavailable, and the device is a prototype. The bridge can enforce honesty rules but cannot validate sensor accuracy.

4. **"How would you secure this for a real network?"** Authentication, authorization, TLS, CSRF/origin controls, rate limits, audit logs, privacy review and a proper deployment architecture. The current localhost-only server is not that system.

5. **"What is the most important backend test?"** The temperature/contact/core/fever rejection suite, because those rules prevent the most serious semantic error: presenting an actual sensor reading or unverified wrist value as a clinical core/fever result.

I.4 Two cross-module questions

1. **"Firmware sends a valid actual TMP117 reading and wrist_contact_status: unverified. What does the bridge accept?"** It accepts the actual fields with matching raw/timestamp/range and requires wrist status unverified with null value/raw/timestamp. It always keeps core null and fever not_available with the exact message.

2. **"How does the dashboard get the history and why can it show gaps?"** /api/history

returns TelemetryStore.history() bucketed points; missing buckets are null. The dashboard's gapAwarePath() starts a new segment rather than drawing through the null, so disconnections are visible.

I.5 Three improvements

1. **Add authenticated/TLS deployment and CSRF protections before any non-local use.** The current localhost-only server is appropriate for a demo, not remote patient data.
2. **Add load/fault-injection tests for serial framing, SSE clients, disk-full/DB-lock and reconnection.** Current tests are synthetic functional tests, not reliability tests.
3. **Add structured diagnostic counters/logging per rejection reason.** invalid_packets exists, but a reason histogram would make field debugging faster and auditable.

I.6 Ten key facts

1. Bridge default: 127.0.0.1:8080; serial 115200; schema 2.
2. STALE_SECONDS = 5.0; stale telemetry is not replayed to a new browser.
3. Duplicate identical serial lines are suppressed for 1.5 s; duplicate JSON keys are always rejected.
4. active_max must be 3; SHT40/MLX present is rejected.
5. Actual TMP117 valid range in the bridge is -55..150 °C; raw must match within 0.02 °C.
6. Unverified wrist contact requires null wrist value/raw/timestamp and status unverified.
7. Core must be null/not_available; fever not_available with exact message.
8. SSE queue size is 8; slow clients drop oldest, not block acquisition.
9. Storage errors are surfaced in hardware_status['storage_error'] but do not stop live telemetry.
10. 62 Python + 10 Node tests pass; physical serial, load and security validation remain incomplete.

I.7 Five terms

Term	Explanation

Term	Explanation
Schema validation	Enforcing the expected types, fields and consistency of the JSON contract before acceptance.
Stale timeout	A time limit after which a packet is no longer considered live; here 5 s at the bridge.
SSE	Server-Sent Events: a one-way HTTP stream used by `/api/stream` to push snapshots/live updates.
Fail-closed	Rejecting or nulling data when evidence is insufficient, rather than guessing.
Defense in depth	Independent checks in firmware, bridge, store and dashboard so one missed check does not silently admit bad data.

I.8 Three source files to open first

1. `telemetry_bridge.py` — serial reader, `parse_packet()`, validators, `snapshot()`, SSE/HTTP handlers.

2. `telemetry_store.py` — `measurements()`, `record()`, `history()`, wrist evidence checks.

3. `tests/test_telemetry.py` — exact backend contract/rejection/stale tests.

I.9 Three things never to claim without evidence

1. **"The live system works end-to-end over real serial/USB."** No device or capture was available in this pass.

2. **"The bridge is secure/network-ready."** It is localhost-only with no authentication/TLS and no security test.

3. **"The bridge validates clinical accuracy."** It validates schema/evidence consistency only; sensor accuracy and clinical validity are unverified.

I.10 Responsibility checklist

- ☐ I can trace serial bytes → line → duplicate check → parse → validation → accept → SSE/store.

- ☐ I can explain `DuplicateSuppressor` versus duplicate JSON keys.

- ☐ I can list the main `_validate_packet()` checks.
- ☐ I can explain actual TMP117 validation and the unverified wrist rule.
- ☐ I can state the exact core/fever constraints and message.
- ☐ I can explain 5-second freshness/no-replay behavior.
- ☐ I can describe SSE queue/drop-oldest behavior.
- ☐ I can name every API route and its failure status.
- ☐ I can explain storage-error handling and why live telemetry continues.
- ☐ I can explain localhost/security limitations.
- ☐ I can cite the fresh test results without calling them hardware validation.
- ☐ I can state the three "never claim" boundaries without prompting.

J. Honest boundaries

J.1 What this member can defend

- The serial reader/framing/reconnect structure.
- The strict schema/status/range validation and its rationale.
- Duplicate and stale handling.
- Actual/wrist/core/fever enforcement in the bridge.
- SSE/API behavior and failure codes.
- Storage handoff and nonfatal storage-error reporting.
- The local-only security boundary.

J.2 What this member must not claim

- That real serial/USB operation or latency was validated in this pass.
- That the bridge is secure for an open network or real patient data.
- That schema validity proves sensor accuracy or clinical validity.
- That the SSE stream guarantees delivery of every packet.

- That storage always succeeds.
- That the bridge computes core temperature, fever or a wrist value.
- That SHT40/MLX90614 are available.
- That the backend uses an AI/LLM model.

J.3 Safe closing sentence

"The backend is implemented as a strict, fail-closed local bridge: it validates the whole contract, expires stale data, keeps actual sensor temperature separate from the unverified clinical wrist field, and serves only fresh accepted packets. The host tests and builds pass, but real serial, load, security and hardware validity still require evidence."